

Gen AI & Agentic AI Interview Questions

Beginner

1. What is Generative AI, explain how it differs from traditional AI.

Answer:

Generative AI refers to a class of artificial intelligence models designed to create new data that resembles a given dataset. These models learn the underlying patterns and structures in the training data and use this knowledge to generate novel outputs such as text, images, music, or code. Popular examples include Large Language Models (LLMs) like GPT, image generators like DALL-E and Midjourney, and audio generators like Jukebox.

Generative AI is fundamentally different from traditional AI in the following ways:

Objective:

- *Traditional AI* is typically discriminative, meaning it focuses on classification, prediction, or decision-making. For example, classifying spam emails, detecting fraud, or recommending products.
- *Generative AI* is generative, focusing on creating content. For example, writing essays, generating synthetic images, or producing code snippets.

Data Modeling:

- *Traditional AI* models estimate $P(y|x)$ — the probability of a label y given input x (e.g., is this image a cat or a dog?).
- *Generative AI* models aim to estimate $P(x)$ or $P(x|\text{context})$ — the probability distribution of the data itself (e.g., generate a realistic-looking cat image or complete a sentence).

Output Type:

- *Traditional AI* produces structured outputs like class labels, scores, or numerical predictions.
- *Generative AI* produces unstructured outputs like paragraphs of text, high-resolution images, or audio streams.

Learning Approach:

- *Traditional AI* typically uses supervised learning, requiring labeled datasets.

- *Generative AI* can use unsupervised, self-supervised, or reinforcement learning methods, often training on massive unlabelled datasets (e.g., the web).

Applications:

- *Traditional AI*: Fraud detection, object recognition, language translation, route optimization.
- *Generative AI*: Content creation (blogs, art), simulation data generation, personalized assistants, drug discovery through molecular design.

In summary, while traditional AI focuses on understanding and making decisions based on existing data, Generative AI emphasizes creating entirely new, human-like data from what it has learned.

2. What are AI agents and what is Agentic AI?

Answer:

AI agents are autonomous systems that perceive their environment, make decisions, and take actions to achieve specific goals. They typically operate using a sense–think–act loop, leveraging tools, memory, and reasoning capabilities.

Agentic AI refers to a paradigm where these agents are designed with more autonomy, goal-orientation, and adaptability, enabling them to operate over long-time horizons, plan dynamically, handle uncertainty, and even delegate sub-tasks to other agents. Agentic AI emphasizes building systems that act with initiative, can self-reflect, revise strategies, and persist across tasks with minimal human intervention.

3. How do Generative AI, AI Agents, and Agentic AI differ? What is their core purpose?

Answer:

Generative AI focuses on generating new content—such as text, images, or code—based on learned patterns from data. Its core purpose is content creation, completion, or augmentation using models like GPT, DALL·E, or Stable Diffusion.

AI Agents are autonomous systems designed to make decisions and perform tasks in an environment. Their purpose is goal-directed execution—using reasoning, planning, and sometimes tools—to solve specific problems.

Agentic AI builds on AI agents by embedding long-term autonomy, self-reflection, tool orchestration, memory, and dynamic planning. Its core purpose is to create systems that act with initiative and adapt across complex, evolving tasks with minimal oversight.

In essence:

- **Generative AI** = content generation
- **AI Agents** = autonomous task execution
- **Agentic AI** = intelligent, self-directed problem-solving systems across time and context

4. What are the main steps to follow in order to build an Agentic AI application?

Answer:

The main steps to build an Agentic AI application are:

- **Define the Agent's Goal & Scope:** Clearly specify the objective, constraints, and success criteria of the agent.
- **Choose a Foundation Model:** Select a capable LLM (e.g., GPT-4, Claude, Gemini) that can perform reasoning, planning, and tool interaction.
- **Design the Agent Architecture:** Incorporate components like:
 - Perception/Input handler
 - Planner (for decomposing tasks)
 - Executor (to perform actions)
 - Memory (short-term and long-term)
 - Reflection & Feedback loop
- **Integrate Tools and APIs:** Allow the agent to interact with external tools (e.g., web search, databases, code execution, or third-party APIs) via tool calling or function calling mechanisms.
- **Implement Memory & Context Management:** Use vector stores or structured storage to persist knowledge across steps and sessions, enabling context awareness and learning over time.
- **Enable Self-Reflection and Re-planning:** Let the agent evaluate past actions, revise plans, or retry failed steps.
- **Test in Simulated and Real Environments:** Validate the agent's decision-making and reliability across edge cases and task variations.

- **Add Guardrails & Safety Layers:** Enforce ethical boundaries, error handling, and user approval checkpoints for critical actions.
- **Deploy and Monitor:** Run the agent in production with observability tools to track performance, failures, and improvement areas.
- **Iterate and Improve:** Continuously fine-tune prompts, memory strategies, and tool orchestration based on feedback and usage data.

5. What are Large Language Models (LLMs), and what is their basic architecture?

Answer:

Large Language Models (LLMs) are Deep Learning Models trained on massive amounts of text data to understand, generate, and manipulate human language. They predict the next word (token) in a sequence, enabling tasks like text generation, translation, summarization, question answering, and more.

Basic Architecture:

- **Transformer Architecture** LLMs are primarily based on the Transformer architecture, which includes:
 - **Input Embedding Layer:** Converts tokens into dense vectors.
 - **Positional Encoding:** Adds information about token positions since Transformers lack recurrence.
 - **Stacked Transformer Blocks:** Each block contains:
 - ✓ **Multi-Head Self-Attention:** Lets the model focus on different parts of the input simultaneously.
 - ✓ **Feedforward Neural Network (MLP):** Processes the attended features.
 - ✓ **Layer Normalization & Residual Connections:** Aid in training stability and gradient flow.
 - **Output Layer:** Maps final hidden states to vocabulary scores for next-token prediction.
- **Autoregressive vs. BERT-style**
 - **Autoregressive LLMs (e.g., GPT):** Predict tokens from left to right (causal).
 - **Autoencoding LLMs (e.g., BERT):** Predict masked tokens in context (bidirectional).
- **Training Objective** Most LLMs use **next-token prediction** (causal language modeling) or **masked language modeling** as their core training task.
- **Scalability** LLMs scale in performance with more parameters, training data, and compute, as shown by scaling laws.
- Popular examples: GPT-3/4, LLaMA, Claude, Gemini, and PaLM.

6. What are some real-world applications of Generative AI and Agentic AI?

Answer:

Generative AI Applications:

- Content creation (text, images, video)
- Code generation (e.g., GitHub Copilot)
- Chatbots and virtual assistants
- Design mockups and prototyping
- Educational tools and tutoring
- Game content and storytelling

Agentic AI Applications:

- Autonomous research (e.g., AutoGPT)
- AI coding agents (e.g., SWE-agent)
- Business workflow automation
- Data analysis and reporting
- Full-resolution customer support agents
- Personal AI assistants (e.g., scheduling, email)
- Robotics and real-world task execution

7. Explain the concept of prompt engineering and describe zero-shot, one-shot, and few-shot prompting.

Answer:

Prompt engineering is the practice of crafting inputs (prompts) to guide large language models (LLMs) to produce desired outputs. It involves structuring instructions, context, and examples to optimize model behaviour for a specific task.

Types of Prompting:

1. Zero-shot Prompting

- No examples are provided.
- The model relies purely on instructions.
- Example: "Translate this to French: 'Good morning.'"

2. One-shot Prompting

- One example is given before the actual task.
- Helps guide the model on format or intent.

- Example: "Translate: 'Hello' → 'Bonjour'. Now translate: 'Good night' →"

3. Few-shot Prompting

- A few examples (typically 2–5) are given.
- Improves accuracy for more complex tasks.
- Example: "Translate: 'Hello' → 'Bonjour'. 'Thank you' → 'Merci'. 'Good night' →"

These methods help LLMs generalize and perform tasks without needing fine-tuning.

8. What are common challenges of using Generative AI?

Answer:

Common challenges of using Generative AI include:

- Hallucination** Models may generate incorrect or fabricated information confidently.
- Bias and Fairness** Outputs can reflect societal, racial, or gender biases present in training data.
- Data Privacy** Models may inadvertently leak sensitive or proprietary information.
- Content Safety** Risk of generating harmful, toxic, or inappropriate content.
- Lack of Interpretability** It's hard to understand or explain how and why models make specific decisions.
- Dependence on Prompt Quality** Output quality is highly sensitive to prompt phrasing and structure.
- Computational Costs** Running and fine-tuning large models requires significant compute and energy.
- Version Drift** Model behaviour can change across updates, affecting reliability.
- Legal and IP Issues** Uncertainty around content ownership, copyright, and commercial use.

9. Describe Retrieval Augmented Generation (RAG) architecture, its purpose, and its key benefits.

Answer:

Retrieval-Augmented Generation (RAG) is an architecture that combines a language model with a retrieval system to improve factual accuracy and context awareness during text generation.

Purpose:

To ground the language model's responses in external knowledge by retrieving relevant documents at inference time, reducing hallucinations and enhancing performance on knowledge-intensive tasks.

Architecture Components:

1. Retriever:

- a. Fetches relevant documents from an external knowledge base (e.g., vector database) based on the user's query.
- b. Typically uses dense embeddings (e.g., from sentence transformers).

2. Generator (LLM):

- a. Takes the original query + retrieved documents as input.
- b. Generates a grounded, contextually relevant response.

3. Optional Components:

- a. **Indexing layer:** Converts documents to vector embeddings for retrieval.
- b. **Re-ranker:** Improves quality by selecting the most relevant documents from retrieved candidates.

Key Benefits:

- **Improved Accuracy:** Uses real-time data instead of relying solely on pretraining.
- **Reduced Hallucination:** Grounds responses in retrieved, verifiable information.
- **Domain Adaptability:** Easily updated by changing the underlying knowledge base without retraining the model.
- **Data Efficiency:** No need to retrain the LLM to incorporate new knowledge.
- **Explainability:** Provides source documents, making responses more transparent.

RAG is commonly used in chatbots, question-answering systems, and enterprise search applications.

10. What are some initial ethical concerns associated with Generative AI and Agentic AI?

Answer:

Ethical Concerns in Generative AI:

1. **Misinformation & Deepfakes** Generation of fake news, realistic images, or videos can spread false narratives.
2. **Bias and Discrimination** Models may reinforce harmful stereotypes present in training data.
3. **Intellectual Property (IP) Violations** Generated content may infringe on copyrighted material.

4. **Privacy Leaks** Risk of unintentionally generating sensitive or personal data seen during training.
5. **Content Safety** Potential to produce toxic, violent, or harmful outputs.

Ethical Concerns in Agentic AI:

1. **Autonomy without Oversight** Highly autonomous agents may act unpredictably or take unintended actions.
2. **Accountability** Unclear who is responsible for the actions or failures of a self-directed agent.
3. **Manipulation or Deception** Agents may exploit loopholes or manipulate users to achieve goals.
4. **Misuse in Automation** Agents could be misused to execute harmful or malicious tasks at scale.
5. **Loss of Human Control** Over-reliance on agents may reduce human oversight in critical decisions.

Addressing these concerns requires ethical design, transparent evaluation, safety mechanisms, and clear accountability frameworks.

Intermediate

1. Explain the transformer architecture and elaborate the attention mechanism

Answer:

Transformer Architecture

The **Transformer** is a deep learning architecture introduced in the paper "*Attention Is All You Need*" (Vaswani et al., 2017). It is designed to process sequences in parallel (unlike RNNs) using a mechanism called **self-attention**, making it highly efficient and scalable.

Core Components:

1. **Input Embedding + Positional Encoding**
 - a. Converts tokens into dense vectors.
 - b. Adds positional information since Transformers lack recurrence.
2. **Encoder–Decoder Structure** (for seq-to-seq tasks):
 - a. **Encoder:** Processes the input sequence.
 - b. **Decoder:** Generates output, one token at a time, using encoder output + previously generated tokens.

(LLMs like GPT use only the decoder stack in causal mode.)

3. **Stacked Transformer Blocks (N layers)** Each block includes:

- a. **Multi-Head Attention**
- b. **Feedforward Network (MLP)**
- c. **Residual Connections + Layer Normalization**

Attention Mechanism (Specifically Self-Attention)

Self-attention allows each token in a sequence to **weigh its relationship with other tokens**, enabling context-aware understanding.

Step-by-Step:

1. **Inputs:** A sequence of embeddings $\rightarrow$ matrix $\mathbf{X}$
 - a. For each token, compute:
 - i. **Query (Q)**
 - ii. **Key (K)**
 - iii. **Value (V)** via learned linear projections.

2. **Attention Scores:**

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- a. QK^T : Measures relevance between tokens.
 - b. $\sqrt{d_k}$: Scales the scores to stabilize gradients.
 - c. **softmax**: Turns scores into attention weights.
 - d. Multiply weights with $\mathbf{V}$ to compute contextualized output.
3. **Multi-Head Attention:**
 - a. Runs multiple attention operations in parallel with different projections.
 - b. Captures diverse relationships (e.g., syntax, semantics) across heads.
 - c. Outputs are concatenated and linearly transformed.

Benefits of Attention:

- Captures long-range dependencies efficiently.
- Allows parallel computation (vs. RNNs).
- Learns contextual representations dynamically.

The **Transformer** architecture, powered by the **attention mechanism**, is foundational for models like BERT, GPT, T5, LLaMA, and many others.

2. What is context window, temperature, top-p, and top-k in LLMs?

Answer:

1. Context Window

The maximum number of tokens a model can consider at once.

- Includes both input and output tokens.
- Determines how much context the model can retain.
- *Example: GPT-4 has a 128k-token context window (for some versions).*

2. Temperature

Controls randomness in output generation.

- **Low (e.g., 0.1–0.3):** More deterministic and focused.
- **High (e.g., 0.7–1.0):** More diverse and creative.
- *Temperature = 0* often produces the same output every time.

3. Top-p (Nucleus Sampling)

Chooses from the smallest possible set of tokens whose cumulative probability $\geq p$.

- **p = 0.9:** Sample from top 90% of probability mass.
- Encourages diversity while filtering unlikely tokens.

4. Top-k Sampling

Selects from the top k most probable tokens.

- **k = 50:** Sample only from the top 50 likely next tokens.
- Lower k = more focused; higher k = more diverse.

3. What is a Vector Database and why use it with LLMs?

Answer:

A **Vector Database** is a specialized database designed to store and index high-dimensional vector representations (embeddings) of data such as text, images, or audio. These vectors are generated using machine learning models and capture semantic meaning, enabling similarity search.

Why use it with LLMs:

1. **Semantic Search:** LLMs convert text into embeddings. A vector database allows efficient retrieval of semantically similar data (e.g., documents, passages) based on these embeddings.
2. **RAG (Retrieval-Augmented Generation):** In RAG systems, vector databases enable fetching relevant context from large knowledge bases to ground LLM responses with accurate, domain-specific information.

3. **Scalability:** Supports approximate nearest neighbor (ANN) search for millions of embeddings with low latency.
4. **Personalization and Memory:** Stores user interactions or preferences as embeddings, enabling personalized or context-aware generation.
5. **Multi-modal support:** Allows cross-modal search (e.g., text query retrieving similar images), useful in AI agents that deal with diverse data.

Popular vector DBs: **Pinecone, Qdrant, Weaviate, FAISS, Milvus.**

4. What is Euclidean distance and Cosine similarity in vector databases?

Answer:

Euclidean Distance and **Cosine Similarity** are metrics used in vector databases to measure similarity or dissimilarity between vector embeddings.

Euclidean Distance:

- Measures the straight-line (L2 norm) distance between two vectors in space.
- Formula:

$$d(\vec{a}, \vec{b}) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

- **Range:** $[0, \infty)$, where 0 means the vectors are identical.
- **Use case:** Works well when the magnitude of the vectors is meaningful.

Cosine Similarity:

- Measures the cosine of the angle between two vectors, focusing on orientation rather than magnitude.
- Formula:

$$\text{cosine_similarity}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|}$$

- **Range:** $[-1, 1]$; commonly $[0, 1]$ for non-negative embeddings, where 1 means vectors point in the same direction.
- **Use case:** Preferred when vector magnitude isn't meaningful, such as with text embeddings from LLMs.

5. What is the purpose of the LangChain framework, and how do its core components like Prompt Templates, Chains, and Output Parsers facilitate LLM application development?

Answer:

LangChain is a framework designed to simplify the development of applications using large language models (LLMs) by providing modular abstractions and integrations. Its purpose is to help developers structure prompts, manage interactions with LLMs, and integrate external tools or data sources efficiently.

- **Prompt Templates** standardize and parameterize prompt construction, allowing dynamic insertion of variables into prompt text. This ensures consistency and reusability across different use cases.
- **Chains** represent sequences of operations—often combining prompts, LLM calls, and outputs—in a pipeline. This abstraction helps in building multi-step workflows such as summarization, question answering, or RAG pipelines.
- **Output Parsers** convert the raw text output from LLMs into structured data (e.g., JSON, Python objects), making downstream processing more reliable and automatable.

Together, these components make LLM applications more composable, maintainable, and production-ready.

6. When might you use a Multi-Agent system?

Answer:

You might use a Multi-Agent system when solving complex tasks that require:

- **Decomposition of subtasks:** Tasks like autonomous research, complex planning, or customer support can be broken down into smaller, specialized roles (e.g., researcher, planner, executor).
- **Role specialization:** Different agents can be designed with unique abilities or prompt-engineered personalities (e.g., code-writer vs. code-reviewer) to collaborate efficiently.
- **Parallelism and scalability:** Agents can work concurrently to explore different solutions, handle multiple users, or process tasks in parallel.
- **Inter-agent collaboration:** When tasks require negotiation, consensus, or information exchange (e.g., multi-turn dialogues, simulation environments, or financial advising).

Examples include:

- Autonomous AI agents performing end-to-end web research and reporting
- Customer service agents routing queries to domain-specific agents

- Simulations of human-like interactions or economic systems
- Multi-modal agents coordinating vision, language, and planning tasks

7. **Discuss the Model Context Protocol (MCP) and the Agent2Agent (A2A) Protocol, explaining their purposes, key capabilities, and how they relate in the evolving AI agent ecosystem.**

Answer:

Model Context Protocol (MCP) is a standard for managing the evolving context of interactions with AI models and agents. It ensures agents maintain memory, track tool usage, preserve conversation history, and retain task-related metadata across sessions. Key capabilities include context persistence, tool invocation tracking, session state management, and support for composability and observability. MCP enables consistent, stateful behavior in single-agent and multi-agent workflows.

Agent2Agent (A2A) Protocol defines a structured communication format between agents. It standardizes how agents express intent, delegate tasks, share results, and negotiate roles. Key capabilities include message schemas with sender/receiver roles, task definitions, capability descriptions, and reasoning traces. A2A enables multi-agent collaboration, dynamic coordination, and decentralized problem-solving.

Relation: MCP handles the internal state and context within agents, while A2A governs external communication between them. Together, they provide the foundation for building scalable, interoperable, and modular agentic systems, where agents can reliably maintain context and interact across complex workflows.

8. **Explain the purpose of LLM fine-tuning, and when should we opt for it instead of RAG?**

Answer:

LLM fine-tuning customizes a pre-trained language model on a domain-specific or task-specific dataset, modifying the model's weights to improve its performance on targeted outputs.

Purpose:

- Adapt the model to a specific domain (e.g., legal, medical, financial)
- Improve task-specific performance (e.g., classification, summarization, code generation)
- Align model behavior with organizational tone, goals, or formats
- Enable generalization in low-context or offline environments (no retrieval)

Opt for fine-tuning when:

- You need consistent performance on narrow, repetitive tasks
- Latency and offline inference are critical (no time for dynamic retrieval)
- Your use case involves structured outputs or specific label sets
- You have high-quality, labeled data for training

RAG (Retrieval-Augmented Generation) is preferable when:

- The knowledge base is large, dynamic, or frequently updated
- You want transparency and traceability of responses
- You have limited or no fine-tuning data
- You need up-to-date or factual grounding from external sources

In summary: use **fine-tuning** for domain-specific behavior baked into the model; use **RAG** for flexible, knowledge-rich, dynamic applications.

9. What is Low-Rank Adaptation (LoRA) in LLM fine-tuning?

Answer:

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning technique for large language models that injects small, trainable low-rank matrices into the model's weight layers instead of updating the full set of parameters.

Purpose:

- Reduce computational cost and memory usage during fine-tuning
- Allow efficient customization of large models without retraining all weights
- Enable multiple task-specific adapters to be stored and swapped easily

How it works:

- For a given weight matrix $W \in \mathbf{R}^{d \times k}$, LoRA freezes W and adds a learnable low-rank decomposition $\Delta W = AB$, where $A \in \mathbf{R}^{d \times r}$, $B \in \mathbf{R}^{r \times k}$, $r \ll \min(d, k)$
- During training, only A and B are updated; at inference, $W + \Delta W$ is used

Benefits:

- Dramatically fewer trainable parameters
- Faster training and lower hardware requirements
- Modular adapters for multi-task or multi-domain scenarios

LoRA is widely used in open-source LLM fine-tuning frameworks like PEFT, Unsloth and Hugging Face Transformers.

10. What is quantization and what is QLoRA?

Answer:

Quantization is a model compression technique that reduces the numerical precision of a model's weights and/or activations—typically from 32-bit floating point (FP32) down to 8-bit integers (INT8) or lower (4-bit, etc.). By representing parameters at lower bit-width, quantization:

- **Shrinks model size** and memory footprint
- **Speeds up inference** due to simpler arithmetic
- **Lowers hardware requirements** (VRAM, bandwidth)

Quantization can be done post-training (static or dynamic quantization) or during training (quantization-aware training) to minimize accuracy loss.

QLoRA (Quantized Low-Rank Adaptation) combines 4-bit quantization of the base large language model with LoRA adapters for efficient fine-tuning. Key elements:

- **4-bit quantized base model:** The frozen backbone is stored in 4-bit, drastically cutting memory usage.
- **LoRA adapters:** Small, trainable low-rank matrices (A , B) are injected into selected layers; only these are updated.
- **Paged optimizers & double quantization:** Further reduce GPU memory by offloading and optimizing parameter storage.

Benefits of QLoRA:

- Enables fine-tuning of 65B+ parameter models on consumer GPUs (e.g., 24 GB VRAM)
- Preserves most of the original model's performance with minimal accuracy drop
- Allows multiple task-specific adapters to be stored and swapped without duplicating the full model

Advanced

1. How can we evaluate the performance of Agentic AI systems?

Answer:

- **Agentic AI systems** require evaluation of not only their final answers but also their multi-step behaviour.
- New metrics have been developed for this. For example, Azure's Agentic Evaluation library defines metrics like Task Adherence (does the final output correctly address the user's

request?), Tool Call Accuracy (are the agent's API/tool calls appropriate and correctly executed?), and Intent Resolution (did the agent understand the user's goal?).

- Task Adherence checks if the answer stays on-topic and complete; Tool Call Accuracy checks procedural correctness when using tools; Intent Resolution checks if the agent's plan matches the user's intent.
- These metrics use LLM evaluators to compare agent outputs with expected behaviour.
- In summary, evaluation combines traditional NLP scores (like BLEU/ROUGE) with specialized, agent-focused scores that examine planning and tool use in the agent's workflow.

2. You have a legal document that's 100 pages long. Explain strategies to get accurate Q&A from an LLM with a 4K-token context window

Answer:

1. **Chunking with Overlap:** Split the document into smaller, semantically coherent chunks (e.g., 300–500 tokens) with overlaps to maintain context continuity.
2. **Embedding-Based Retrieval (RAG):** Convert chunks into vector embeddings and store them in a vector database. On receiving a query, retrieve the most relevant chunks using semantic similarity.
3. **Top-k Context Injection:** Select top-k relevant chunks (e.g., 3–5) that fit within the 4K-token window and insert them alongside the query into the prompt for answer generation.
4. **Prompt Engineering:** Use structured prompts like "Answer the question based only on the following context..." to ensure answers stay grounded in retrieved content.
5. **Summarization for Compression:** Pre-summarize lengthy sections into compact representations that retain legal specificity, allowing more content to fit in the context window.
6. **Hierarchical QA:** For broad questions, decompose into sub-questions, answer them individually using context-aware retrieval, then synthesize a final response.
7. **Answer Verification:** Use a second LLM pass to validate the answer against retrieved context, ensuring traceability and factual grounding.

3. Explain how you'd design metadata filters in ChromaDB (or other similar vectorDB) to let users search a legal corpus by jurisdiction and case type.

Answer:

To design metadata filters in ChromaDB for searching a legal corpus by **jurisdiction** and **case type**, I would follow these steps:

1. **Document Ingestion with Metadata:**
 - a. While adding documents to ChromaDB, I would include relevant metadata alongside the embeddings:


```
collection.add(
    documents=["..."],
    embeddings=[...],
    metadatas=[{"jurisdiction": "US-Federal", "case_type":
"Criminal"}],
    ids=["doc_001"]
)
```

2. Standardize Metadata Schema:

- a. Define consistent keys and controlled vocabularies:
 - i. jurisdiction: e.g., "US-Federal", "US-State-California", "India-Supreme"
 - ii. case_type: e.g., "Criminal", "Civil", "Family", "Tax"

3. Enable Metadata Filtering in Queries:

- a. During a similarity search, use the where clause to filter:

```
collection.query(
    query_embeddings=[...],
    n_results=5,
    where={"jurisdiction": "US-Federal", "case_type":
"Criminal"}
)
```

4. Advanced Filtering (Optional):

- a. For multi-valued filters or partial matches, structure metadata accordingly:

```
where={"jurisdiction": {"$in": ["US-Federal", "US-State-
NewYork"]}}
```

5. Index Design Considerations:

- a. Choose a vector DB (e.g., Qdrant, Weaviate, ChromaDB) that supports metadata indexing for faster filtered queries.

This setup ensures that semantic search results are scoped only to documents matching the user's legal domain context.

4. You need to generate product descriptions for 1,000 SKUs. Walk me through how you'd design your prompts to ensure consistent tone and accuracy.

Answer:

1. **Define the desired tone and style:** Create a style guide specifying tone (e.g., professional, casual, friendly), length, grammar preferences, keywords to include, and what to avoid (e.g., exaggeration).
2. **Template-based prompting:** Create structured prompts using consistent templates. Example:

Generate a product description for a [product_type].

Brand: {brand_name}

Features: {feature_list}

Use-case: {use_case}

Target audience: {audience}

Tone: {tone_style}

Word limit: {limit}

3. **Use metadata per SKU:** For each SKU, pass its structured metadata (title, brand, specs, features, category, usage) into the template.
4. **Few-shot prompting:** Include 2–3 well-crafted examples in the prompt that reflect the tone and structure I want. This trains the model implicitly to maintain consistency.
5. **Chunking for batching:** Group SKUs by category (e.g., "kitchen appliances") to reuse prompts and style context efficiently.
6. **Validation loop:** Design prompts with output parsers or schema constraints (e.g., JSON output with fields: title, features, benefits, CTA) to validate model output.
7. **Human-in-the-loop QA:** Sample-check outputs for tone drift, factuality, and formatting. Fine-tune prompts or the LLM if inconsistencies emerge at scale.

5. When building a Retrieval-Augmented Generation system, what are the pros and cons of fetching long passages vs. short passages from your knowledge base?

Answer:

Long Passages:

Pros:

- More context per chunk, reducing chances of losing important information.
- Fewer retrievals required, improving latency and reducing token usage for embedding lookup.

Cons:

- May include irrelevant information that dilutes relevance and confuses the LLM.
- Higher chance of exceeding context window limits, especially with multiple documents.
- Harder to score precisely during retrieval due to mixed-topic content.

Short Passages:

Pros:

- More focused and relevant results, improving answer precision.

- Easier to rank and retrieve based on semantic similarity.
- Enables hybrid retrieval (e.g., combining semantic + keyword match).

Cons:

- Risk of losing essential context spread across multiple chunks.
- Requires more retrievals, increasing latency and token overhead.
- May demand chunk reassembly logic during generation for coherence.

6. Imagine you're building a customer-support chatbot. How would you decide whether to use a traditional rule-based system or a generative model like an LLM? What trade-offs would you consider?

Answer:

Decision Factors:

1. Complexity of Queries:

- Rule-based:* Suitable if queries are repetitive, structured, and well-defined (e.g., "What's my order status?").
- LLM:* Better for handling open-ended, ambiguous, or multi-turn queries.

2. Coverage of Use Cases:

- Rule-based:* Requires manual flow design; limited scalability.
- LLM:* Can generalize across unseen queries using existing knowledge.

3. Accuracy and Control:

- Rule-based:* High precision and full control over responses; low hallucination risk.
- LLM:* Less predictable, may hallucinate or give partially incorrect responses without grounding.

4. Maintenance and Scalability:

- Rule-based:* Costly to update as rules grow; rigid logic.
- LLM:* Easier to scale with less manual intervention; adaptable via fine-tuning or RAG.

5. Latency and Cost:

- Rule-based:* Fast and cheap.
- LLM:* Higher latency and inference cost.

6. Compliance and Risk:

- Rule-based:* Safer in regulated domains due to deterministic behavior.
- LLM:* Requires guardrails (e.g., output filtering, retrieval grounding) to ensure compliance.

Trade-Off: If coverage breadth, natural language flexibility, and scalability are priorities, use an LLM (possibly with RAG and guardrails). If precision, predictability, and cost are more critical, start with a rule-based system or a hybrid design.

7. With generative and agentic AI evolving rapidly, how would you architect your platform to accommodate new LLM variants, protocols, and regulatory changes over the next 2–3 years?

Answer:

- i) **Modular LLM Abstraction Layer:**
 - a. Implement a provider-agnostic interface (e.g., using LangChain, LlamaIndex, or custom adapters) to swap LLMs (OpenAI, Anthropic, open-source) without rewriting core logic.
 - b. Support both API-based and local inference (e.g., vLLM, GGUF).
- ii) **Pluggable Agent Framework:**
 - a. Use agent frameworks (e.g., CrewAI, LangGraph) designed for composability and orchestration.
 - b. Architect agents as independent, stateful services with defined roles, tools, and memory.
- iii) **Retrieval Layer with Index Abstraction:**
 - a. Decouple vector DBs using retrieval interfaces; support multiple backends (e.g., Qdrant, Weaviate, Elasticsearch) to adapt to scaling or compliance needs.
 - b. Maintain separate indexes for fast swap/experimentation.
- iv) **Dynamic Prompt/Tool Management:**
 - a. Store prompts and toolchains in versioned config stores (e.g., YAML or DB) to allow A/B testing, rollback, and adaptation to new model capabilities or instructions.
- v) **Observability and Guardrails:**
 - a. Instrument all LLM and agent responses with tracing, feedback collection, and flagging systems.
 - b. Integrate moderation, red-teaming, and output validation pipelines (e.g., Guardrails.ai, Rebuff).
- vi) **Regulatory & Compliance Layer:**
 - a. Maintain data lineage, consent tracking, and model usage logs.
 - b. Ensure auditability by logging model input/output, retrieval context, and decision paths.
- vii) **Governance via Policy Engines:**
 - a. Use policy-as-code (e.g., OPA) to enforce usage boundaries, fallback logic, and user access controls based on regulatory shifts.
- viii) **CI/CD for AI Workflows:**
 - a. Automate validation of new model integrations, agent upgrades, or tool changes using eval frameworks and canary deployments.
- ix) **Future-proofing:**
 - a. Track open standards (e.g., OpenAI Function Calling, JSON mode, Open Agents Protocol) and adopt specs that ensure interoperability across ecosystems.

8. Explain how 4-bit quantization can impact both inference latency and model accuracy. In which scenarios might you avoid aggressive quantization?

Answer:

Impact on Inference Latency:

- **Reduced Memory Footprint:** 4-bit quantization significantly reduces model size, enabling faster memory access and cache usage.
- **Improved Throughput:** Smaller weights allow more parallelism and lower bandwidth demands, improving latency, especially on GPU/TPU/accelerator hardware optimized for low-bit operations.
- **Hardware Dependency:** Gains depend on hardware support for 4-bit math; without it, performance may degrade due to emulation overhead.

Impact on Accuracy:

- **Precision Loss:** Aggressive quantization (e.g., 4-bit) can lead to accuracy degradation, especially in sensitive layers or smaller models.
- **Task Sensitivity:** Tasks like text classification or summarization may tolerate minor loss, but reasoning-heavy tasks (e.g., code generation, math) often suffer more.
- **Model Type:** Larger models are typically more robust to quantization due to overparameterization.

When to Avoid Aggressive Quantization:

- **High-Stakes or Regulated Domains:** Where even minor hallucinations or misclassifications are unacceptable (e.g., healthcare, finance, legal).
- **Complex Reasoning Tasks:** Where accuracy degradation significantly affects output utility.
- **Low-Resource Models:** Small models are less resilient to aggressive quantization.
- **Inference on Incompatible Hardware:** When the underlying compute doesn't support efficient 4-bit operations, leading to performance regressions.

In such cases, consider mixed-precision (e.g., 8-bit with selective 16-bit) or quantization-aware training for better trade-offs.

9. Your company has a proprietary dataset of 10,000 support transcripts. Would you choose LoRA, QLoRA, or full-model fine-tuning? Justify your decision in terms of cost, performance, and maintainability.

Answer:

I would choose **QLoRA**.

- **Cost:** QLoRA enables fine-tuning large models on a single GPU by quantizing weights to 4-bits and using paged optimizers, significantly reducing memory and compute requirements compared to full fine-tuning. It's more cost-efficient than LoRA when fine-tuning larger base models.
- **Performance:** QLoRA supports full model adaptability while still allowing for effective generalization. It retains the benefits of LoRA's parameter-efficient approach while enabling deeper tuning with quantized weights, which is beneficial for adapting to the nuances in support transcript data.
- **Maintainability:** Like LoRA, QLoRA is modular—low-rank adapters are injected without changing the base model's architecture, simplifying version control, rollback, and deployment. It also allows combining multiple QLoRA adapters for different use cases.

In contrast, **full-model fine-tuning** is prohibitively expensive, harder to maintain, and risks overfitting on a relatively small dataset. **LoRA** is a viable alternative, but QLoRA offers better memory efficiency and scalability without sacrificing much in performance.

10. Your enterprise uses Google Drive, Jira, and Confluence. Sketch how you'd build an MCP server to let any LLM-based assistant query across those systems.

Answer:

To build an MCP (Multi-Channel Platform) server that enables LLM-based assistants to query Google Drive, Jira, and Confluence, I would architect it as follows:

1. High-Level Architecture

- **LLM Layer:** Interfaces with the assistant, handles natural language understanding.
- **MCP Server:** Acts as an orchestrator, routing and resolving user intents across services.
- **Connectors Layer:** Service-specific API wrappers for Google Drive, Jira, and Confluence.
- **Semantic Index Layer (optional):** Vector DB for semantic retrieval across documents.
- **Authentication & Permissions Layer:** Enforces OAuth2 and access control policies per user.

2. Components & Flow

A. LLM Request Handling

- The assistant receives a query like: *"What's the latest design doc for Project X and who's assigned the next sprint task?"*
- The request is sent to the **MCP Server** with context/session metadata.

B. Intent & Entity Resolution

- The MCP server:
 - Parses the request via prompt templating + LLM or rules-based classifier.
 - Identifies sub-intents:
 - Query Google Drive (design doc)
 - Query Jira (task assignment)

C. Connectors Implementation

Each connector includes:

- **Google Drive Connector**
 - Uses Google Drive API to search and fetch docs.
 - Optional: integrates with a vector DB for semantic search across document contents.
- **Jira Connector**
 - Uses Jira REST API to fetch sprint board, tickets, assignee data.
- **Confluence Connector**
 - Searches pages via REST API or GraphQL for documentation or project notes.

D. Response Aggregation

- MCP merges responses:
 - Summarizes or reformats into a unified natural language response.
 - Optionally uses the LLM again for output synthesis.

3. Infra & Maintainability

- **Auth:** OAuth 2.0 token flow for Google + Atlassian; stored securely per user.
- **Queue/Worker** (optional): For long-running or batch queries.
- **Logs & Traces:** Full observability to monitor per-service latency and failures.
- **Rate Limiting & Caching:** Avoid over-querying APIs, cache frequent lookups.

4. Tools

- Use **LangChain / LlamaIndex** for chaining and retrieval logic.
- Use **FastAPI or Node.js** for the MCP server.
- Use **Qdrant or Weaviate** if semantic search across Drive/Confluence is needed.

This architecture allows flexible expansion (e.g., add Slack or GitHub later), clear separation of concerns, and secure, observable multi-system orchestration for LLM assistants.

Codebasics